

SQL-I

Assoc. Prof. Kitsana Waiyamai, Ph. D.
Dept. Of Computer Engineering
Faculty of Engineering, Kasetsart University
Bangkok, Thailand



Example: College Admission Database

College(cName,state,enrollment)

Student(sID,sName,GPA,sizeHS)

Apply(sID,cName,major,decision)

	cName	state	enrollment
1	Stanford	CA	15000
2	Berkeley	CA	36000
3	MIT	MA	10000
4	Cornell	NY	21000

	sID	sName	GPA	sizeHS
1	123	Amy	3.9	1000
2	234	Bob	3.6	1500
3	345	Craig	3.5	500
4	456	Doris	3.9	1000
5	567	Edward	2.9	2000
6	678	Fay	3.8	200
7	789	Gary	3.4	800
8	987	Helen	3.7	800
9	876	Irene	3.9	400
10	765	Jay	2.9	1500
11	654	Amy	3.9	1000
12	543	Craig	3.4	2000

	sID	cName	major	decision
1	123	Stanford	CS	Y
2	123	Stanford	EE	N
3	123	Berkeley	CS	Y
4	123	Cornell	EE	Y
5	234	Berkeley	biology	N
6	345	MIT	bioengineering	Y
7	345	Cornell	bioengineering	N
8	345	Cornell	CS	Y
9	345	Cornell	EE	N
10	678	Stanford	history	Y
11	987	Stanford	CS	Y
12	987	Berkeley	CS	Y
13	876	Stanford	CS	N
14	876	MIT	biology	Y
15	876	MIT	marine biology	N
16	765	Stanford	history	Y
17	765	Cornell	history	N
18	765	Cornell	psychology	Y
19	543	MIT	CS	N

SQL: Expression, Rename result attribute

```
/*****************************************
  Add scaled GPA based on sizeHS
  Also note missing Where clause
*****************************************

select sid, sname, GPA, sizeHS,
GPA*(sizeHS/1000.0)
from Student;

/*** Rename result attribute ***/

select sid, sname, GPA, sizeHS,
GPA*(sizeHS/1000.0) as scaledGPA
from Student;
```

	sID	sName	GPA	sizeHS	scaledGPA
1	123	Amy	3.9	1000	3.9
2	234	Bob	3.6	1500	5.3999996
3	345	Craig	3.5	500	1.75
4	456	Doris	3.9	1000	3.9
5	543	Craig	3.4	2000	6.8
6	567	Edward	2.9	2000	5.8
7	654	Amy	3.9	1000	3.9
8	678	Fay	3.8	200	0.76
9	765	Jay	2.9	1500	4.3500004
10	789	Gary	3.4	800	2.72
11	876	Irene	3.9	400	1.5600001
12	987	Helen	3.7	800	2.96

$\rho_{\text{sid, sname, GPA, sizeHS, scaledGPA}}(\pi_{\text{sid, sname, GPA, sizeHS, GPA*(sizeHS/1000.0)}}(\text{Student}))$

SQL: %, * operators

```
/*************************************************************
  All applications to bio
*************************************************************/
select sID, major
from Apply
where major like '%bio%';

/*** Same query with Select * ***/
select *
from Apply
where major like '%bio%';
```

	sID	major
1	234	biology
2	345	bioengineering
3	345	bioengineering
4	876	biology
5	876	marine biology

	sid	cname	major	decision
1	234	Berkeley	biology	N
2	345	Cornell	bioengineering	N
3	345	MIT	bioengineering	Y
4	876	MIT	biology	Y
5	876	MIT	marine biology	N

SQL: Cartesian (cross-product)

```
/*************************************************
   Select * cross-product
*************************************************
select *
from Student, Apply;
```

Student × Apply

Theta-Join

Theta -Join

$$Exp_1 \bowtie_\theta Exp_2 \equiv \sigma_\theta (Exp_1 \times Exp_2)$$

condition

- Theta-join: Condition can be <, <=, >, >=, <>, =

- Basic operation implemented in DBMS
  => Term "**join**" often means **Theta -Join**

SQL: Theta-Join

	sName	major
1	Amy	CS
2	Amy	EE
3	Amy	CS
4	Amy	EE
5	Bob	biology
6	Craig	bioengineering
7	Craig	CS
8	Craig	EE
9	Craig	bioengineering

```
/*************************************************************
  Student names and majors for which they've applied
*************************************************************/
select sName, major
from Student, Apply
where Student.sID = Apply.sID;

/*** Same query with Distinct, note difference from algebra ***/
```

- We got set of students & set of majors they apply for
- In SQL => There is duplicates !!!!!
- In RA **=> The DISTINCT is automatically handled by the projection (π).**

```
select distinct sName, major
from Student, Apply
where Student.sID = Apply.sID;
```

$$\pi_{\text{sName, major}}(\text{Student} \bowtie_{\text{Student.sID = Apply.sID}} \text{Apply})$$

SQL: Aggregation

```
Select A1,A2,…,An
From   R1,R2, …,Rm
Where  condition
```

"Aggregation" functions over values in multiple rows: **min**, **max**, **sum**, **avg**, **count**

Demo: simple college admissions database

College(cName,state,enrollment)

Student(sID,sName,GPA,sizeHS)

Apply(sID,cName,major,decision)

SQL: Aggregation

```
/*************************************************************
  Highest GPA Among Students
*************************************************************/
select max(gpa) as max_gpa
from student;
```

```
/*************************************************************
  Average GPA of all students
*************************************************************/
select avg(GPA)
from Student;
```

Aggregation in Relational Algebra

G avg(GPA) (Student)

G (gamma) is the **aggregation operator** in relational algebra

avg(GPA) specifies the aggregate function (average) applied to the GPA attribute

SQL: Aggregation

```
/*********************************************************
  Number of colleges bigger than 15,000
*********************************************************/
select count(*)
from College
where enrollment > 15000;
```

SQL: count(*) operators

```
/*************************************************************
  All applications to bio
*************************************************************/
select sID, major
from Apply
where major like '%bio%';
```

	sID	major
1	234	biology
2	345	bioengineering
3	345	bioengineering
4	876	biology
5	876	marine biology

```
/*** Same query with Select * ***/
select *
from Apply
where major like '%bio%’;
```

	sid	cname	major	decision
1	234	Berkeley	biology	N
2	345	Cornell	bioengineering	N
3	345	MIT	bioengineering	Y
4	876	MIT	biology	Y
5	876	MIT	marine biology	N

```
/*** Same query with count(*) ***/
select count(*) as total_number
from Apply
where major like '%bio%';
```

	total_number
1	5

SQL: Table Variables

```
Select A1,A2,…,An
From   R1,R2, …,Rm
Where  condition
```



SQL: Table Variables

```
/*************************************************************
  Student names and majors for which they've appli
*************************************************************/
select distinct sName, major
from Student, Apply
where Student.sID = Apply.sID;
```

	sName	major
1	Amy	CS
2	Amy	EE
3	Amy	CS
4	Amy	EE
5	Bob	biology
6	Craig	bioengineering
7	Craig	CS
8	Craig	EE
9	Craig	bioengineering

π sName, major (Student $\bowtie$ Student.sID = Apply.sID Apply)

```
select distinct sName, major
from Student s, Apply a
where s.sID = a.sID;
```

$\pi_{\text{sName, major}} (\sigma_{\text{s.sID = a.sID}} (\rho_{\text{s (Student)}} \times \rho_{\text{a (Apply)}}))$

Exercise-I

Exercise-II

Exercise-III

SQL is an unordered model

```
/*************************************************************
  Application information
*************************************************************/

select Student.sid, sname, GPA, Apply.cname, enrollment
from Student, College, Apply
where Apply.sid = Student.sid and Apply.cname = College.cname;
```

	sID	sName	GPA	cName	enrollment
1	123	Amy	3.9	Berkeley	36000
2	123	Amy	3.9	Cornell	21000
3	123	Amy	3.9	Stanford	15000
4	123	Amy	3.9	Stanford	15000
5	234	Bob	3.6	Berkeley	36000
6	345	Craig	3.5	Cornell	21000
7	345	Craig	3.5	Cornell	21000
8	345	Craig	3.5	Cornell	21000
9	345	Craig	3.5	MIT	10000
10	543	Craig	3.4	MIT	10000
11	678	Fay	3.8	Berkeley	36000

SQL: Order by

```
/*********************************************************
  Application information Sort by decreasing GPA, increasing enrollment

select Student.sid, sname, GPA, Apply.cname, enrollment
from Student, College, Apply
where Apply.sid = Student.sid and Apply.cname = College.cname
order by GPA desc, enrollment;
```

	sID	sName	GPA	cName	enrollment
1	876	Irene	3.9	MIT	10000
2	876	Irene	3.9	MIT	10000
3	876	Irene	3.9	Stanford	15000
4	123	Amy	3.9	Stanford	15000
5	123	Amy	3.9	Stanford	15000
6	123	Amy	3.9	Cornell	21000
7	123	Amy	3.9	Berkeley	36000
8	678	Fay	3.8	Stanford	15000
9	678	Fay	3.8	Berkeley	36000

SQL: Table Variables

```
/*********************************************************
  Application information
*********************************************************/

select Students.s_id, s_name, GPA, Aply.c_name, enrollment
from Students, Colleges, Aply
where Aply.s_id = Students.s_id and Aply.c_name = Colleges.c_name;
/*** Introduce table variables ***/

select S.s_id, S.s_name, S.GPA, A.c_name, C.enrollment
from Students S, Colleges C, Aply A
where A.s_id = S.s_id and A.c_name = C.c_name;
```

	sID	sName	GPA	cName	enrollment
1	876	Irene	3.9	MIT	10000
2	876	Irene	3.9	MIT	10000
3	876	Irene	3.9	Stanford	15000
4	123	Amy	3.9	Stanford	15000
5	123	Amy	3.9	Stanford	15000
6	123	Amy	3.9	Cornell	21000
7	123	Amy	3.9	Berkeley	36000
8	678	Fay	3.8	Stanford	15000

Exercises-IV

SQL: Set Operators

Select $A_1, A_2, \ldots, A_n$

From $R_1, R_2, \ldots, R_m$

Where condition

Set Operators:
Union, Intersect, Except

SQL: Set Operators
Union => No-duplicates

```
/************************************
  List of college names and
student names
************************************


select cName from College
union
select sName from Student;


*** Add 'As name' to both sides


select cName as name from College
union
select sName as name from
Student;
```

	name
1	Amy
2	Berkeley
3	Bob
4	Cornell
5	Craig
6	Doris
7	Edward
8	Fay
9	Gary
10	Helen
11	Irene
12	Jay
13	MIT
14	Stanford

SQL: Set Operators
Union All => Duplicates

```
/*************************************************************
  List of college names and student names
*************************************************************/

/*** Change to Union All ***/ => with duplicates

select cName as name from College
union all
select sName as name from Student;

/*** Add order by cName ***/

select cName as name from College
union all
select sName as name from Student
order by name;
```

	name
1	Amy
2	Amy
3	Berkeley
4	Bob
5	Cornell
6	Craig
7	Craig
8	Doris
9	Edward
10	Fay
11	Gary
12	Helen
13	Irene
14	Jay
15	MIT
16	Stanford

SQL: Set Operators => Intersect

```
/*************************************************************
  IDs of students who applied to both CS and EE
*************************************************************/

select sID from Apply where major = 'CS'
intersect
select sID from Apply where major = 'EE';

/*************************************************************
  IDs of students who applied to both CS and EE
  Some systems don't support intersect
*************************************************************/
```

Exercises-V

SQL: Set Operators => except

```
/*********************************************************
IDs of students who applied to CS but not EE
*********************************************************/

select sID from Apply where major = 'CS'
except
select sID from Apply where major = 'EE';

/*********************************************************
  IDs of students who applied to both CS and EE
  Some systems don't support except
*********************************************************/
```

SQL: Aggregation

```
Select A1,A2,…,An
From   R1,R2, …,Rm
Where  condition
```

"Aggregation" functions over values in multiple rows: **min**, **max**, **sum**, **avg**, **count**

SQL: Aggregation Group by

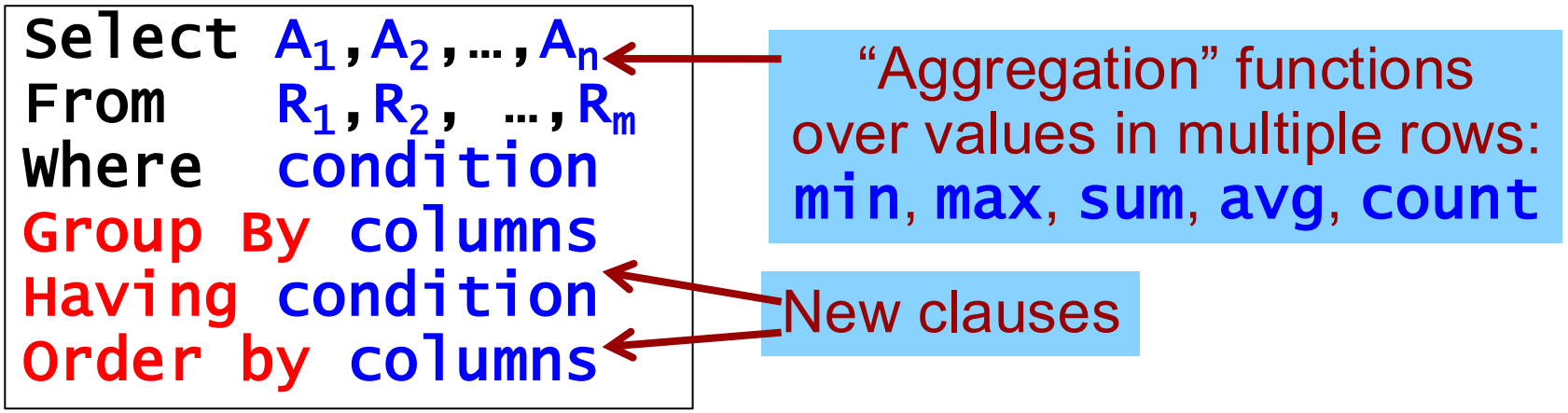
```
/*********************************************************
    Applications ordered by name
   *********************************************************/
  select *
  from Apply
  order by cname

/*********************************************************
    Number of applications to each colleg
   *********************************************************/
  select cname, count(*) as …
  from Apply
  group by cname
```

	sid	cname	major	decision
1	123	Berkeley	CS	Y
2	234	Berkeley	biology	N
3	678	Berkeley	CS	N
4	987	Berkeley	CS	Y
5	765	Cornell	history	N
6	765	Cornell	psychology	Y
7	345	Cornell	bioengineering	N
8	345	Cornell	CS	Y
9	345	Cornell	EE	N
10	123	Cornell	EE	Y
11	345	MIT	bioengineering	Y

	cName	number_applications
1	Berkeley	4
2	Cornell	6
3	MIT	4
4	Stanford	6

SQL: Group-By


Select $A_1, A_2, \ldots, A_n$
From $R_1, R_2, \ldots, R_m$
Where condition
Group By columns
Having condition
Order by columns
"Aggregation" functions over values in multiple rows: min, max, sum, avg, count
New clauses

SQL: Aggregation Group by

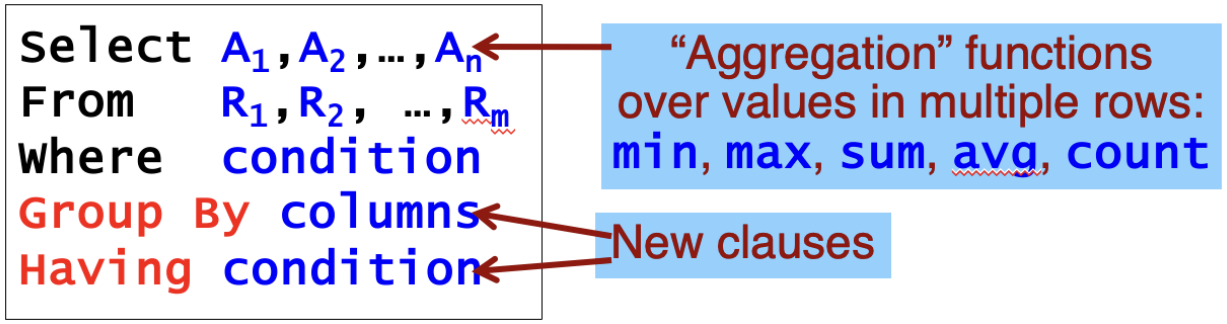
```
/*********************************************************
    Number of applications to each college, major
  *********************************************************/
  select cname, major, count(*) as nb_of_applications
  from Apply
  group by cname;
```

- MySQL does not know **which major value** to show for each cname
- Every selected attribute that is not inside an aggregate function should normally appear in GROUP BY.

```
  select cname, major, count(*) as nb_of_applications
  from Apply
  group by cname, major;
```

SQL: Group-By

- Perform computations over sets of values in multiple rows
- Basic aggregation functions supported by every SQL system are
  - **Minimum,**
  - **Maximum,**
  - **Sum,**
  - **Average,**
  - **Count.**



- Group by allows us to partition our relations into groups and then will compute aggregated aggregate functions over each group independently.
- Having condition allows us to test filters on the results of aggregate values.
- Having condition will apply to the groups that we generate from the group by clause.

SQL: Aggregation Group by, Having

```
/*********************************************************
  College enrollments by state name (sum of enrollment by state)
*********************************************************

select state, sum(enrollment)
from College
group by state;


/*********************************************************
  College names with fewer than 5 applications
*********************************************************/
select cname, count(*)
from Apply
group by cname
having count(*) < 5
```

- Every selected attribute that is not inside an aggregate function should normally appear in GROUP BY.

SQL: Aggregation Group by, Having

```
/*************************************************************
  applications for each college + major having
*************************************************************/
select cname, major, count(*) as nb_of_applications
from Apply
group by cname, major
Order by cname;
```

- Every selected attribute that is not inside an aggregate function should normally appear in GROUP BY.

SQL: Group-By

- Perform computations over sets of values in multiple rows
- Basic aggregation functions supported by every SQL system are minimum, maximum, some, average and count.
- Group by allows us to partition our relations into groups and then will compute aggregated aggregate functions over each group independently.

- Having condition will apply to **the groups** that we generate from the group by clause.
- Where condition applies to **single rows** at a time.

SQL: Aggregation Group by

```
/*****************************************************
 For each student with his name, number of applications
*****************************************************/
select Student.sID, sName
from Student, Apply
where Student.sID = Apply.sID
order by Student.sID;
/*****************************************************
  Solution
*****************************************************/
select Student.sID, sName, count(Student.sID) as number_of_apps
from Student, Apply
where Student.sID = Apply.sID
group by Student.sID, sName;
```

- Every selected attribute that is not inside an aggregate function should normally appear in GROUP BY.

Exercises-VI

SQL: Aggregation Group by

```
/*****************************************************
  Minimum + maximum GPAs of applicants to each college & major
*****************************************************/

select cName, major, GPA
from Student, Apply
where Student.sID = Apply.sID
order by cName, major;

/*** Solution ***/

select cName, major, min(GPA), max(GPA)
from Student, Apply
where Student.sID = Apply.sID
group by cName, major;
```

cName	major	min(GPA)	max(GPA)
Berkeley	CS	3.7	3.9
Cornell	EE	3.5	3.9
Stanford	CS	3.7	3.9
Stanford	EE	3.9	3.9
Berkeley	biology	3.6	3.6
Cornell	bioengineering	3.5	3.5
Cornell	CS	3.5	3.5
MIT	bioengineering	3.5	3.5
MIT	CS	3.4	3.4
Stanford	history	2.9	3.8

Exercises-VII